

Reviewer Pack — Minimal Quasi-Morphism Rank and Communication Complexity Low...

Entry 2f6c61e4b2c3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 16:27:05 UTC

Minimal Quasi-Morphism Rank and Communication Complexity Lower Bound

Entry ID: 2f6c61e4b2c3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 16:27:05 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Quasi-morphisms) **Field B** (complexity object): Communication Complexity (Deterministic Communication Complexity)

Statement:

For every Boolean function f with n input bits, the minimal rank of a quasi-morphism φ_f that preserves f is linearly correlated with its deterministic communication complexity $c(f)$, such that $r(\varphi_f) = \Theta(c(f))$.

Rationale (proposer's reasoning):

Quasi-morphisms are a generalization of morphisms in algebraic topology and can be used to study the structure of Boolean functions. The minimal rank of a quasi-morphism φ_f provides a measure of the complexity of f , which might reveal new insights into communication complexity lower bounds. This conjecture aims to bridge the gap between algebraic topology and communication complexity by proposing a quantitative relationship between these two areas.

Taxonomy category: Quasi-morphism rank (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 490a1a4790197f14

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated Boolean functions with n input bits ($n \leq 40$), the correlation coefficient between minimal quasi-morphism rank $r(\varphi_f)$ and deterministic communication complexity $c(f)$ across 30 random seeds is at least 0.8, and there is no seed where $|r(\varphi_f) - c(f)| > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quasi-morphism rank" AND "communication complexity" AND deterministic - "Boolean function" AND "minimal quasi-morphism rank" AND communication complexity - "preserving Boolean functions" AND quasi-morphism rank AND deterministic communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_boolean_function(n):
    return [random.choice([0, 1]) for _ in range(2**n)]

def compute_deterministic_communication_complexity(f):
    n = int(math.log2(len(f)))
    complexity = 0
    for i in range(n):
        count_0 = f[:2**(i+1)].count(0)
        count_1 = f[:2**(i+1)].count(1)
        if count_0 > 0 and count_1 > 0:
            complexity += 1
    return complexity

def quasi_morphism_rank(f):
    n = int(math.log2(len(f)))
    if n <= 1:
        return 0

    def is_quasi_morphism(r, f):
        for i in range(n):
            for j in range(i+1, n):
                if f[i] != f[j]:
                    if abs(r[i] - r[j]) > 1:
                        return False
        return True
```

```

min_rank = n
for rank in range(1, n + 1):
    for perm in itertools.permutations(range(n), rank):
        r = [0] * n
        for i in range(n):
            if i not in perm:
                r[i] = f[perm.index(i)]
        if is_quasi_morphism(r, f):
            min_rank = rank
            break
    if min_rank < n:
        break

return min_rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        f = generate_random_boolean_function(n)
        r = quasi_morphism_rank(f)
        c = compute_deterministic_communication_complexity(f)

        if len(results) >= 30:
            break

        results.append({
            "n": n,
            "r": r,
            "c": c
        })

    if len(results) < 30:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(result["n"] for result in results),
            "conjecture_holds": False,
            "counterexample": "not_enough_instances"
        }

    r_values = [result["r"] for result in results]
    c_values = [result["c"] for result in results]

    mean_r = sum(r_values) / len(r_values)
    mean_c = sum(c_values) / len(c_values)

    correlation_coefficient = 0
    numerator = sum((r - mean_r) * (c - mean_c) for r, c in zip(r_values, c_values))
    denominator = math.sqrt(sum((r - mean_r)**2 for r in r_values)) * math.sqrt(sum((c - mean_c)**2 for c in c_values))
    correlation_coefficient = numerator / denominator

```

```

if denominator == 0:
    correlation_coefficient = None
else:
    correlation_coefficient = numerator / denominator

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": correlation_coefficient is not None and abs(correlation_coefficient) > 0.99,
    "counterexample": "" if correlation_coefficient is not None else "correlation_coefficient < 0.99"
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all("conjecture_holds" not in result or result["conjecture_holds"] for result in results):
        mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = sum(1 for result in results if "conjecture_holds" not in result or result["conjecture_holds"]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=NA support_fraction={support_fraction}")
    elif any("counterexample" in result and result["counterexample"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if "counterexample" in result)
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no_conjecture_holds_or_counterexample")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a92e226.py", line 119, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a92e226.py", line 67, in run_trial
    r = quasi_morphism_rank(f)
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9a92e226.py", line 50, in quasi_morphism_rank
    r[i] = f[perm.index(i)]
            ~~~~~
ValueError: tuple.index(x): x not in tuple

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete the required computations to verify the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and provide results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14723
2	propose	ollama_remote	glm4:latest	0	0	12603
3	propose	ollama_remote	glm4:latest	0	0	13183
4	propose	ollama_remote	glm4:latest	0	0	12368
5	preregistration	ollama_remote	glm4:latest	0	0	11250
6	novelty	ollama_remote	glm4:latest	0	0	8598
7	novelty	ollama_remote	glm4:latest	0	0	11968
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47040
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11998
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10741
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12940
12	judge	ollama_remote	glm4:latest	0	0	14706

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 182120 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in research/audit/2f6c61e4b2c3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/2f6c61e4b2c3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/2f6c61e4b2c3.tar.gz (if generated)